# DSP in VLSI

## Homework 1

### Sorter


**Student ID**

M11407439

**Name**

MING-HONG, LIN


**Development Environment**

| | |
|---|---|
| **HDL** | Verilog HDL |
| **EDA Tool** | Vivado |
| **Verification** | Python |

Due Date: 2026/03/18

# Contents


**1 Procedure 1 — Testbench Generation** **2**
1.1 Testbench Input Generation . . . . . 2

**2 Procedure 2 — Sort8 Implementation** **3**
2.1 Timing Diagram . . . . . 3

**3 Procedure 3 — SelectTopK Implementation** **4**
3.1 Block Diagram and Calculation . . . . . 5
3.2 Timing Diagram and Verification . . . . . 6

**4 Procedure 4 — Full Timing Verification** **7**
4.1 Timing Diagram . . . . . 7

**5 Procedure 5 — Python Verification** **8**
5.1 Verification Following Procedure 4 . . . . . 8

**6 Procedure 6 — Synthesis Report** **9**
6.1 Component Count . . . . . 9

# Procedure 1 — Testbench Generation

First, generate several random sequences with 32 elements $[x_1\, x_2\, \ldots\, x_{32}]$ in a block between $-256$ and 255 as your test inputs and prepare as the testbench. **(10%)**

## 1.1 Testbench Input Generation

For design and verification convenience, instead of generating random sequences externally via Python or MATLAB, random values in the range of $-256$ to 255 are generated directly within the testbench using the expression **($random & 9'h1FF) - 256**, and simultaneously written to **Input.dat** for subsequent Python-based sorting result verification.

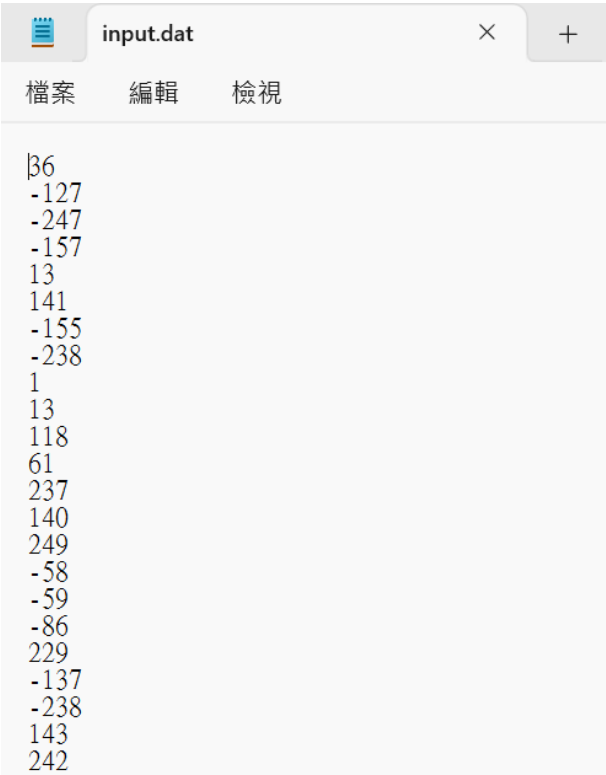


Figure 1.1: Content of Input.dat

# Procedure 2 — Sort8 Implementation

Please use Verilog to implement **Sort8** with bitonic sorter revised from Fig. 4. Feed the 8 inputs $[x_1\, x_2\, \ldots\, x_8]$ simultaneously in one clock cycle. Observe the outputs to realize sorting among eight elements in one group. Use $[x_1\, x_2\, \ldots\, x_8]$ as the input and check for the correctness of the function "**Sort8**", which can generate sorted output from maximum to minimum. Indicate this function output in the timing diagram of your behavior simulation results. (If the data value inside the timing diagram can not be clearly seen, it will not be regarded as correct answers.) **(10%)**

## 2.1 Timing Diagram

In this Module, due to the excessive combinational comparator path length causing a large critical path delay, two pipeline registers are introduced to reduce the propagation delay per stage, resulting in an output latency of two clock cycles.

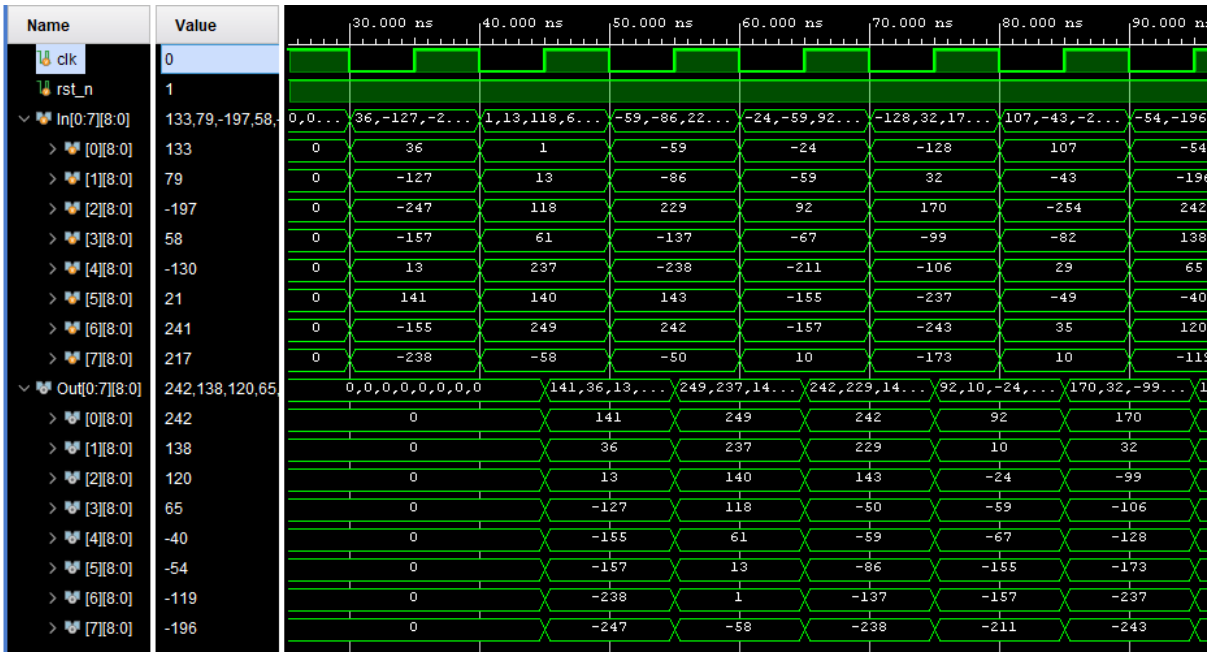


Figure 2.1: Timing Diagram of Sort8 Behavior Simulation Results

# Procedure 3 — SelectTopK Implementation

Construct the module (**SelectTopK**) of merger sort to select top 4 among 32 input elements ($[x_1\ x_2\ x_3\ \ldots\ x_{32}]$) under your control. One example is given as in Fig. 5. Use a comparator tree to receive the sorted results from each group and then activate your comparator tree to select top 4 values. Maybe you need two sets of **Reg8** buffers, one for saving the outputs of the next block from Sort8, and the other for sending the contents of the current block to the comparator tree.

- Draw the block diagram of your own implementation and calculate the number of adders/subtractors/comparators in your block diagram. **(20%)**
- Show the timing diagram of your behavior simulation results. **(10%)**
- Show the hardware output is correct by comparing it to your Matlab (Python) results (It means you need to provide the correct answer from the command "sort" by yourself). **(10%)**

## 3.1 Block Diagram and Calculation

The Sort8 module uses a total of **24** comparators, and the bottom-level comparator stage requires an additional **3**, bringing the overall comparator count to **27**. As for adders, **3** are used : one for the Reg Counter that determines which register group to store the Sort8 output into, one for the Pointer adder, and one for the OutRank adder. No subtractors are required.

| Comparators | Adders | Subtractors |
|---|---|---|
| 27 | 3 | 0 |

Table 3.1: Component Count of SelectTopK Module

In this module, a Ping-Pong Buffer is employed to maintain high throughput. The first three register groups adopt a full Ping-Pong mechanism, while the last group writes directly to the Pong Buffer, as its data arrival timing naturally aligns with the output timing and no conflict arises. This enables continuous output across different blocks without additional latency. Furthermore, since only the top-4 values are required, each Ping and Pong Buffer requires only 4 registers.

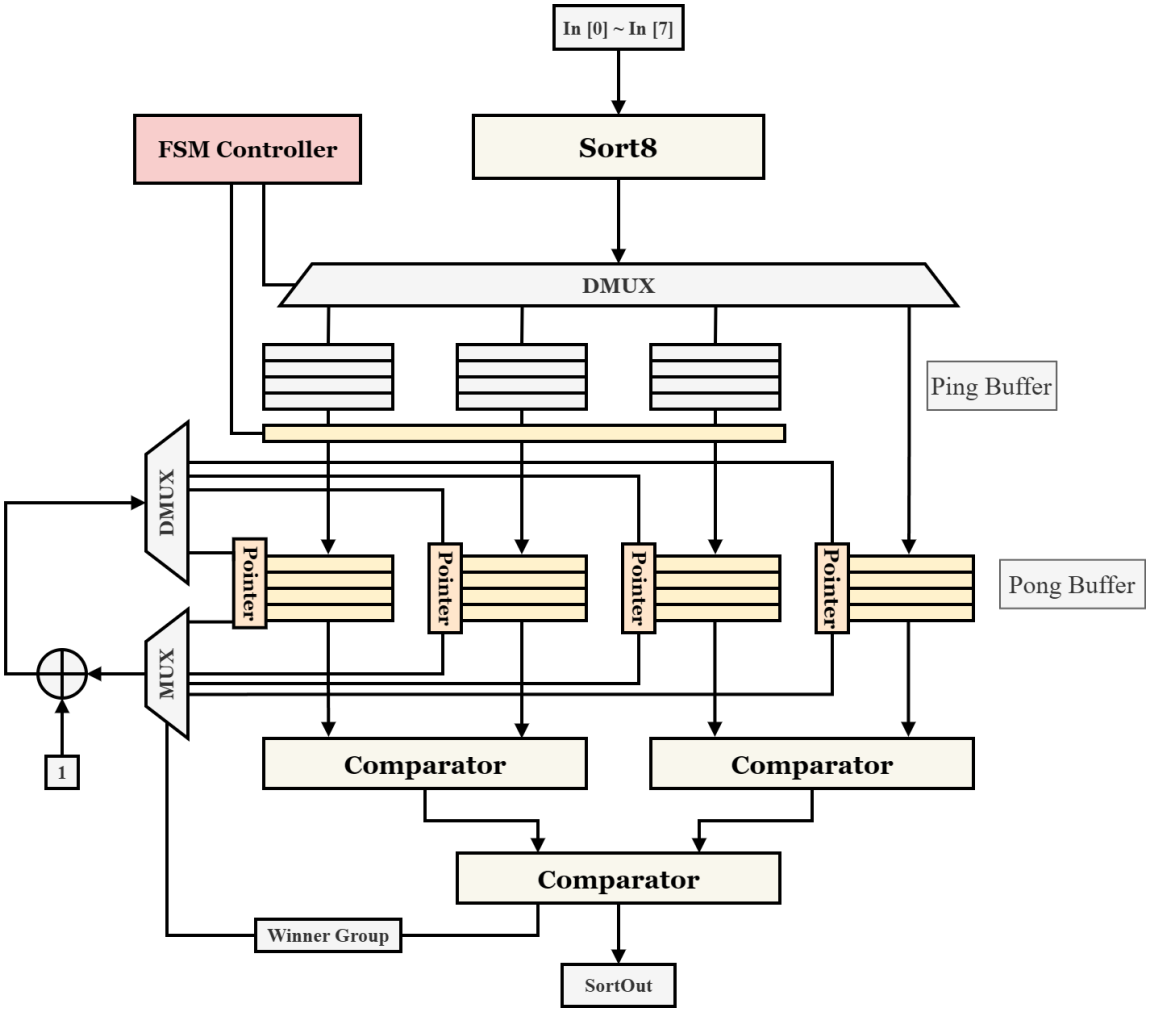


Figure 3.1: Block Diagram of SelectTopK

## 3.2 Timing Diagram and Verification

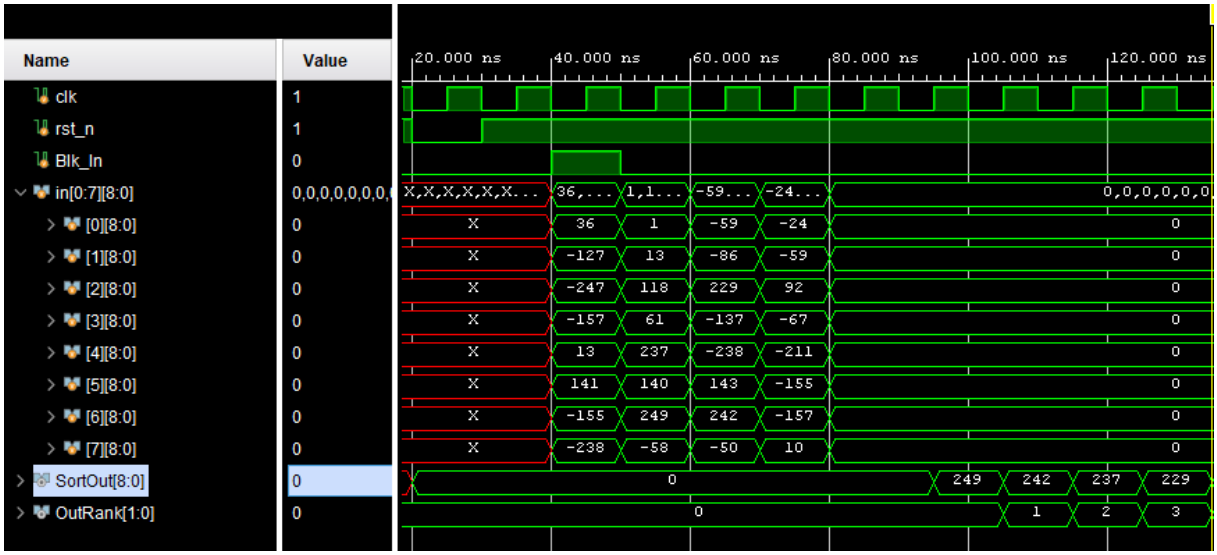


Figure 3.2: Timing Diagram of SelectTopK Behavior Simulation Results

```
PS D:\DSP-In-VLSI\HW1-Sorter> python .\find_top4.py
Group 1: [249, 242, 237, 229]
PS D:\DSP-In-VLSI\HW1-Sorter>
```

Figure 3.3: Sorting Results by Python

# Procedure 4 — Full Timing Verification

Show that your design can generate required top-4 output in descending order every 4 clock cycles as indicated in Fig. 3 in the timing diagram with proper input signal "**BlkIn**" and output indicator "**OutRank**". Please feed 4 blocks and check the outputs of four blocks. If it is too long, please cut it down into several segments to make the numerical expressions in the timing diagram clear. If the numerical expressions are hard to distinguish, correct evaluation may not be given. **(30%)**

## 4.1 Timing Diagram

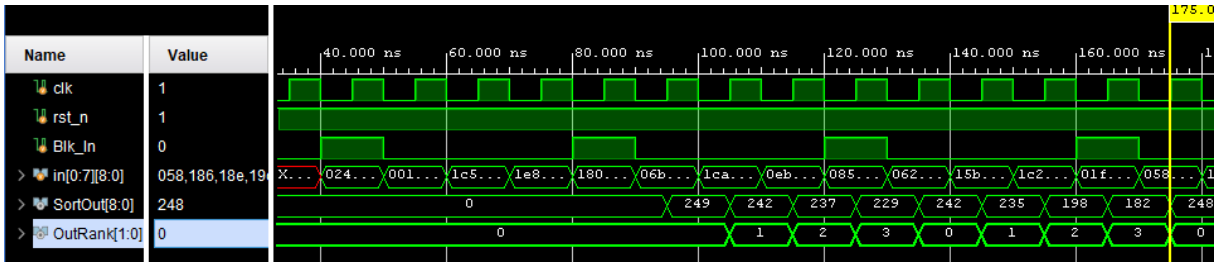


Figure 4.1: Timing Diagram of SelectTopK Behavior Simulation Results

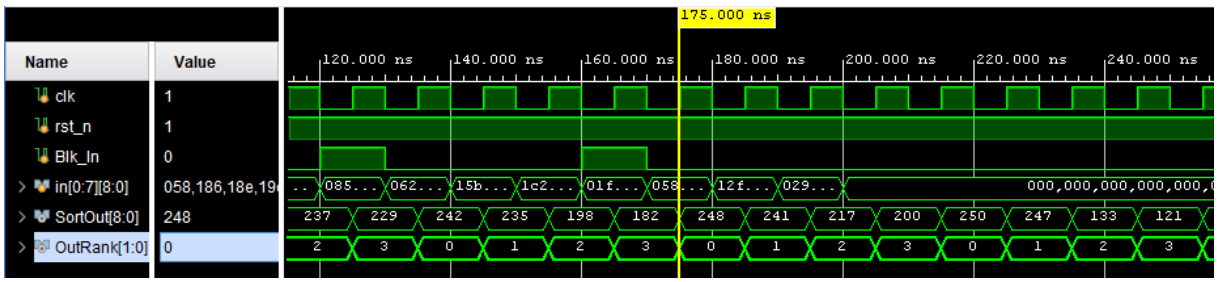


Figure 4.2: Timing Diagram of SelectTopK Behavior Simulation Results (Continued)

# Procedure 5 — Python Verification

Please use Matlab (Python) command "sort" to verify your results of your randomly-generated sequence and compare to the Verilog simulation results. **(10%)**

## 5.1 Verification Following Procedure 4

The sorting results generated by Python **are identical to** the simulation results from the Verilog simulation in Procedure 4.

```
PS D:\DSP-In-VLSI\HW1-Sorter> python .\find_top4.py
Group 1: [249, 242, 237, 229]
Group 2: [242, 235, 198, 182]
Group 3: [248, 241, 217, 200]
Group 4: [250, 247, 133, 121]
PS D:\DSP-In-VLSI\HW1-Sorter>
```

Figure 5.1: Sorting Results by Python

# Procedure 6 — Synthesis Report

Synthesis your design in Procedure 4. Show the number of adders/subtractors/comparators in your design. Sum them up together to see if it matches with your block diagram. **(10%)** (Because you may use counters to control your circuits, we will not ask that the two values should be exactly the same. However, there should not be a large discrepancy.)

## 6.1 Component Count

The component counts after synthesis are exactly identical to the estimates from Procedure 3.

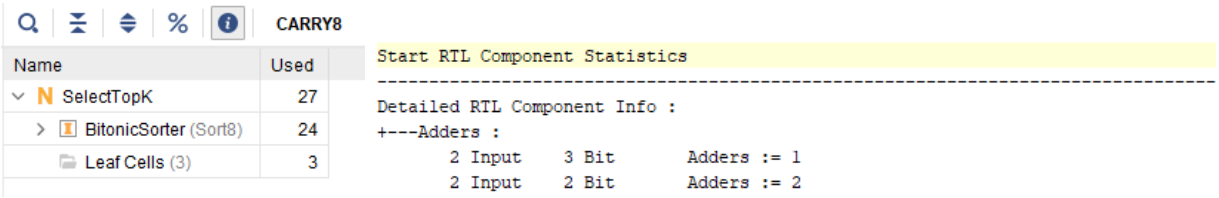


Figure 6.1: Component Counts Report

| Comparators | Adders | Subtractors |
|---|---|---|
| 27 | 3 | 0 |

Table 6.1: Component Count Summary